Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

CS101

Incremental implementation

Today
Welcome to your next lesson: Incremental implementation.

I'm preparing your personalized lesson based on what you've already learned, what's coming up, and the best way to build on your skills.

Step by step, you’re moving forward.

Imagine we’re asked to build a shopping cart shipping-fee helper in Python 🚚

Feature request (in plain English):

“Given a cart total, return the shipping fee:

If total is under 50 → fee is 10
If total is 50 or more → fee is 0”

In this lesson, instead of writing everything at once, we’ll build it little by little, and after each tiny step we’ll run a quick check to see if that step works.

Before we touch any code, I want to feel where you are:

Which way feels more natural to you right now?

A) Write the whole feature (all rules) first, then test once at the end
B) Write and test one tiny behavior at a time (many small tests)

You don’t have to justify it yet — just pick the one that feels like you.

b

Nice, B is exactly the mindset for this lesson 🎯

Let’s start super small.

Imagine we only care about one rule for now:

“If total is under 50 → fee is 10”

We’ll ignore the “50 or more → 0” part for the moment.

So our first tiny behavior is:

Input: total = 30

Expected output: fee = 10

We'll write one function in Python later to handle this. For now, focus on the idea:

This (30 → 10) pair is what I'll call a smallest testable unit: one input, one expected output.

Does that idea make sense: pick just one clear input and one correct output as our first target?

Yes, that makes sense

Great 👍

Now let's connect that idea to how we build in Python.

We'll follow this incremental workflow (one tiny slice at a time):

Pick one behaviour (our (30 → 10) case).
Write a very simple function for it in Python.
Write one quick check (using assert or print) to see if the function gives 10 for 30.
Only after that works, we add the next behaviour.

Visually, the loop looks like this:

code a bit → check quickly → code a bit → check quickly …

We won't write any Python yet; I just want the loop clear.

Are you okay if our very first goal is only this:

"Make a function that, when given 30, returns 10 (we won't care what it does for any other number yet)?"

Yes, that's fine

Awesome 😄

Let's turn that simple goal into code, one piece only.

We'll create a function called calculate_shipping_fee that, for now, only needs to work for total = 30 and should return 10.

Step by step plan:

In the code editor, create a function calculate_shipping_fee(total).

Inside, for now, you can even hard-code it to always return 10.

Under the function, add a quick check like:

```
result = calculate_shipping_fee(30)
print(result)  # we expect to see 10
```

We're not making it "smart" yet — we just want a tiny success and to see the micro-test idea in action.

Go ahead and do that in the code editor (not in chat):

Create calculate_shipping_fee, always return 10, call it with 30, and Submit code.

I'll look at the output with you and we'll take the next step from there 👣

Python

```
def calculate_shipping_fee(total):
    return 10
result = calculate_shipping_fee(30)
print(result)  # we expect to see 10
```

Nice work, Ron — that's exactly the first step ☑

You wrote one function, tried one input, and saw the expected output: 10.

This is the micro□verification loop in action:

Write a tiny bit of code (calculate_shipping_fee).
Run a tiny check (call it with 30, print the result).
Confirm the behavior matches your expectation.
Next small upgrade: instead of just seeing 10, we'll have Python check it for us with an assert.

Idea:

Python

```
result = calculate_shipping_fee(30)
assert result == 10
print("Test passed!")
```

If the function returns anything other than 10, Python will raise an error. If not, you'll see "Test passed!".

Can you update your code in the editor to:

Keep your function as□is,
Replace the print(result) with the assert pattern above,
Then run and Submit code again?
Python

```
def calculate_shipping_fee(total):
    return 10
result = calculate_shipping_fee(30)
assert result == 10
print("Test paassed")  # we expect to see 10
```

Beautiful 🎯

You just did a full micro□loop:

Wrote a tiny function.
Checked one specific case with assert.
Got a "green" signal (Test paassed printed).

Now, let's stretch the function a bit.

Right now:

Python

```
def calculate_shipping_fee(total):
    return 10
```

This returns 10 even if total is 80, which is not what we want long□term.

Next tiny behaviour we want:

If total is 50 or more, fee should be 0.

So our new smallest testable unit will be:

Input: total = 80 → Expected: fee = 0

We'll add one more assert for that, and then adjust the function so both asserts pass.

Step 1 (in your head):

Does this new test make sense as our next tiny goal?

calculate_shipping_fee(80) should give 0.

If yes, I'll have you add a second assert as the next step.

Python

```
def calculate_shipping_fee(total):
    return 10
result = calculate_shipping_fee(30)
assert result == 10
print("Test paassed")  # we expect to see 10
```

Yes, that new test is exactly what we want next ☑

Right now your function always returns 10, so:

calculate_shipping_fee(30) → 10 ☑
calculate_shipping_fee(80) → 10 ✗ (we want 0)

Let's do this incrementally again:

In the editor, under your existing assert result == 10, add:

```
result = calculate_shipping_fee(80)
assert result == 0
```

Keep the print("Test paassed") at the end.

Run and Submit code like that, without changing the function yet.
It should fail — and that's good, it shows the second behavior isn't implemented yet.

After we see it fail, we'll adjust the function so both tests pass, one step at a time 💡

Python

```
def calculate_shipping_fee(total):
    return 10
result = calculate_shipping_fee(30)
assert result == 10
result = calculate_shipping_fee(80)
assert result == 0
print("Test paassed")  # we expect to see 10
```

Perfect, Ron — this is exactly what we want right now: a failing test ✖

You've got:

Python

```
def calculate_shipping_fee(total):
    return 10

result = calculate_shipping_fee(30)
assert result == 10          # passes

result = calculate_shipping_fee(80)
assert result == 0           # fails (got 10 instead)
```

This shows clearly:

Behaviour for 30 is done.
Behaviour for 80 (and all totals ≥ 50) is not done yet.

Now we do the incremental upgrade: change the function so it handles both rules:

If total < 50 → 10
Else (total >= 50) → 0

Step by step:

In the editor, update calculate_shipping_fee to use an if:

```
def calculate_shipping_fee(total):
    if total < 50:
        return 10
    else:
        return 0
```

Keep your two asserts and the print("Test paassed") exactly as they are.
Run and Submit code again.

Do that change in the code editor and send the new submission — we'll check that both micro-tests now go green ☑☑

Python

```
def calculate_shipping_fee(total):
    if total < 50:
        return 10
    else:
        return 0

result = calculate_shipping_fee(30)
assert result == 10
result = calculate_shipping_fee(80)
assert result == 0
print("Test paassed")  # we expect to see 10
```

What do you think? Pick the option that best matches the "one tiny case at a time"

approach 👇

You just used a micro-verification loop to grow calculate_shipping_fee from a hard-coded return to a correct two-branch rule. Suppose we now want to extend the feature to also handle a "free shipping for VIP customers" rule. Which is the best next smallest testable unit to add?

A.

Add a big set of tests for many totals and both VIP and non-VIP customers all at once, then start changing lots of code until they pass.

B.

Pick one new case like: for a VIP with total 20, the fee should be 0, and write a single new assert for just that case first.

C.

Rewrite the whole function from scratch to handle VIPs and hope the existing tests still pass, then test manually by printing results.

Exactly right 🎯

Choosing one new case like “VIP with total 20 should pay 0” and adding a single new assert is exactly how incremental implementation should feel:

You keep all existing behaviour safe (your current tests stay green).
You add one tiny expectation for the new feature.
You then change just enough code so that:
Old tests still pass ☑
The new test also passes ☑

That’s the micro-verification loop: one small behaviour → one small test → one small code change.

Want to try a similar incremental build on a different little feature next (to really lock this in), or do you prefer to keep extending this shipping example a bit more?

Let's try a new feature

Nice, new feature it is 👌

We’ll keep the same incremental style, just with a different problem.

Here’s a simple, real□world style feature:

Discount helper

Given an original price and a discount percentage, return the final price.

Example rule: “If price is 100 and discount is 10%, result should be 90.”

We’ll not build all the rules at once. First, we pick one smallest testable unit:

Input: price = 100, discount_percent = 10

Expected: final_price = 90

Then the same loop:

write tiny function → add one assert for this case → make it pass → then grow.

Quick check:

Are you okay making this our first target case: (100, 10) → 90, like we did with 30 → 10 before?

Yes, let's start with that

Great 👍

We'll repeat the same pattern you just used.

Step-by-step plan:

In the code editor, create a new function:

```
def apply_discount(price, discount_percent):
    return price  # temporary, just a stub
```

Under it, add one micro-test:

```
result = apply_discount(100, 10)
assert result == 90
print("Test passed")
```

Don't worry that it will fail at first — that's what we want.
First we get a red test, then we change the function to make it green.

Go ahead and set that up in the editor (function + one assert + print), then Submit code so we can look at the failing test together 👣

Python

```
def apply_discount(price, discount_percent):
    return price  # temporary, just a stub
result = apply_discount(100, 10)
assert result == 90
print("Test passed")
```

Perfect, Ron — this is the good kind of failure ☑

Right now:

Python

```
def apply_discount(price, discount_percent):
    return price  # temporary, just a stub

result = apply_discount(100, 10)
assert result == 90   # fails (we got 100)
```

So the test is telling us:

"You haven't implemented the real behavior yet."

Next tiny step: make just this one test pass.

The math we need is:

final price = price minus (price × discount_percent / 100)

So for 100 and 10:

100 - (100 × 10 / 100) = 100 - 10 = 90

Your turn in the editor:

Change only the function body to use that formula:

```
def apply_discount(price, discount_percent):
    discount_amount = price * discount_percent / 100
    final_price = price - discount_amount
    return final_price
```

Keep the assert and print("Test passed") exactly as they are.
Run and Submit code again.

Do that update and send the new run — we'll check the now-green test and then add a second tiny case like we did with shipping.

Python

```
def apply_discount(price, discount_percent):
    discount_amount = price * discount_percent / 100
    final_price = price - discount_amount
    return final_price  # temporary, just a stub

result = apply_discount(100, 10)
assert result == 90
print("Test passed")
```

Nice, Ron — that's a clean green test ☑

You just did:

One tiny case: (100, 10) → 90
One assert to capture it
One code change to make it pass

That's exactly the incremental pattern.

Now I want to check you really "own" the idea of smallest testable unit.

Think about this new feature:

"Score to letter grade" helper

90–100 → "A"
80–89 → "B"
70–79 → "C"

If you were building this incrementally, which one specific case would you choose as your first smallest testable unit to write an assert for?

You can answer in plain English, like:

"Score 95 should give 'A'" or whatever you'd pick.

70

Nice choice 👍

You basically picked a case like:

“If the score is 70, the result should be 'C'.”

That’s a valid smallest testable unit: one clear input, one clear expected output.

If we were coding this, we’d start with something like:

```
assert score_to_grade(70) == "C"
```

Then we’d make the function pass that one test before adding more (like 85 → "B", 95 → "A").

Quick check so I know where to go next with you:

Which part feels most useful to keep practising right now?

Using assert as tiny checks after each function change
Deciding what the “smallest testable unit” is for a new feature
Building a multi-step feature (2–3 functions) one function at a time
using assert

Good focus choice 👍 — assert is your main tool in this lesson.

Let’s zoom in on what assert is doing in plain terms.

In Python:

Python

```
assert some_expression
```
means:

“If some_expression is True, do nothing.

If it’s False, stop the program and raise an AssertionError.”

Example from your discount code:

Python

```
result = apply_discount(100, 10)
assert result == 90
```
If result is 90 → expression result == 90 is True → program continues ☑
If result is not 90 → expression is False → Python raises AssertionError ✖

So each assert is a tiny yes/no test for one behavior.

Does that mental model make sense:

assert is basically “this must be True, or stop”?

Yes, that makes sense

Great 👌

Let’s lock in how you use assert when you’re building something step by step.

Pattern you’ve already used:

Call the function with a concrete input.
Store the result in a variable.
Assert the exact value you expect.

Example with your discount helper:

Python

```
result = apply_discount(100, 10)
assert result == 90
```

This is nicer than just print(result) because:

print shows you something, but doesn’t say if it’s right.
assert encodes the rule in code: “for 100 and 10%, it must be 90”.
Want to try a tiny assert practice together?

Pick one of these and tell me which you’d write first as an assert:

A) apply_discount(200, 50) should give 100
B) apply_discount(50, 0) should give 50
C) apply_discount(80, 25) should give 60
b

Nice pick 💡

apply_discount(50, 0) -> 50 is a great next assert because:

It’s simple math (no discount changes the price).
It checks that your function handles the edge case where the discount is zero.
It’s one clear input → one clear expected output (perfect smallest testable unit).

If we turned your choice into code, it would look like:

Python

```
result = apply_discount(50, 0)
assert result == 50
```

Want to add this new assert to your existing discount code in the editor and submit it, so we can see both tests pass together?

Python

```
def apply_discount(price, discount_percent):
    discount_amount = price * discount_percent / 100
    final_price = price - discount_amount
    return final_price  # temporary, just a stub

result = apply_discount(100, 10)
```

```
assert result == 90

result = apply_discount(50, 0)
assert result == 50

print("Test passed")
```

What do you think the answer is? Pick one option in the question box below 👇

You've just used assert to grow your discount helper one tiny step at a time. In that process, what was the smallest testable unit you kept adding?

A.

A whole set of different inputs at once, checked only by reading printed output.

B.

One specific input (like 100, 10) with one exact expected result, encoded in a single assert.

C.

The entire business rule description in comments, without any concrete values.

Exactly right 🎯

You've been adding one specific input with one exact expected result, each captured in a single assert. That's the core of incremental implementation:

Each assert = one tiny promise: "For this input, the function must return this value."
You then change just enough code to keep all those promises true.

That's how you avoid big, messy debugging later — you grow the behavior in small, verified steps.

You've now:

Built features one function at a time.
Used assert as micro-tests after each change.
Identified and implemented smallest testable units (like (100, 10) → 90).

From your side, does this feel solid enough to move on to the next lesson, or do you want one more short coding exercise with assert before we wrap this one?

I'm ready to move on

Nice work, Ron 🎉

You've nailed the core of incremental implementation:

You built features one function at a time (calculate_shipping_fee, apply_discount).
After each change, you ran tiny verification checks with assert, not just print.
You learned to define a smallest testable unit as:
one specific input → one exact expected output → one assert.

That "code a bit → assert a bit → code a bit → assert a bit" loop is exactly how solid software gets built.

As you go into the next lesson, keep this habit:

Any time you add or change behavior, ask:
"What's one concrete example I can turn into an assert right now?"

You're building like a real engineer now — small, safe, and verified ☑

```
def calculate_shipping_fee(total):
```

```
    if total < 50:
        return 10
    else:
        return 0

result = calculate_shipping_fee(30)
assert result == 10
result = calculate_shipping_fee(80)
assert result == 0
print("Test paassed")  # we expect to see 10
Test paassed

def apply_discount(price, discount_percent):
    discount_amount = price * discount_percent / 100
    final_price = price - discount_amount
    return final_price  # temporary, just a stub

result = apply_discount(100, 10)
assert result == 90
print("Test passed")
Test passed
```